FC UNREAL ENGINE 5.8 DIRECTIVES

안티그래비티 환경 종합 아키텍처 보고서

[제3권] FC 프로젝트 언리얼 5.8 실전 엔지니어링, 모듈러 룰셋 및 자율 셀프 힐링 루프

ENGINE VERSION	CORE TENET	PIPELINE RUNNER	PUBLICATION DATE	ARCHITECT
Unreal Engine 5.8 C++20	Single-Player is Local Multiplayer	Scripts/pipeline.py (Tiered)	2026-10-22	Lead Systems & AI Architect

1. 프로젝트 아키텍처 비전: "Single-Player is Local Multiplayer"

FC 프로젝트는 데디케이티드/리슨 서버 복제 아키텍처 위에 구축된 하이파이 싱글플레이어 경험을 핵심 목표로 한다. 싱글플레이어 모드는 엔진 내부에서 **로컬 리슨 서버(Standalone Client + Local Listen Server)**로 구동되며, 향후 협동(Co-op) 멀티플레이어 모드와의 **100% 기능 패리티(Feature Parity)**를 보장한다.

[최고 아키텍처 원칙] 싱글플레이어 전용 지름길(Shortcuts) 절대 금지 Strict Violation Rule

클라이언트 인스턴스에서 서버 권한 확인 없이 인벤토리를 직접 변이시키거나, 체력을 수정하거나, 아이템을 직접 스폰하는 단독 실행 전용 우회 코드는 엄격히 금지된다. 모든 게임플레이 변이는 반드시 서버 권한(Server Authority)을 통과해야 한다.

2. 4계층 권한(4-Tier Authority) 아키텍처 개요

[FC 4-Tier Authority Separation Model]

1. Server Authority (서버 권한 전용)
- AGameModeBase, AAIController, 전투/피해 판정, 인벤토리 변이
- HasAuthority() 강제, Cooked Asset SSOT (Direct Disk I/O 금지)

2. Replicated State (서버-클라이언트 상태 전파)
- AGameStateBase, APlayerState, Actor Components
- DOREPLIFETIME_CONDITION(COND_OwnerOnly), RepNotify 콜백 바인딩

3. Autonomous Client (입력 및 예측 이동)
- APlayerController, ACharacter (Local Autonomous Proxy)
- Enhanced Input, CMC 클라이언트 예측, Server_* RPC (WithValidation)

4. Simulated Client & UI (프레젠테이션 티어)
- AHUD, UUserWidget (UMG/Slate), Niagara, Audio
- 제로 서버 결합, OnRep 델리게이트 수신 및 MVVM ViewModel 렌더링

3. 안티그래비티 에이전트의 거버넌스 역할

안티그래비티는 C++ 코드를 작성하거나 리팩토링할 때 위의 4계층 권한 경계를 엄격히 감시한다. 클라이언트 RPC에 게임플레이 변이가 포함되거나, Slate/UMG 헤더가 서버 클래스에 포함되는 즉시 도메인 규칙(.antigravity/rules/)과 파이프라인 검증기(pipeline.py static)가 작동하여 오류를 사전에 차단한다.

1. 4계층 권한 분리의 실전 구현 표준

1. Server Authority (서버 권한 전용)

- **역할:** 전투 판정, 체력 가감, 몬스터 스폰, 퀘스트/아이템 변이 등 모든 authoritative 게임 상태 결정.
- **코드 보장:** if (HasAuthority()) 또는 if (GetLocalRole() == ROLE_Authority) 강제.
- **Cooked Asset SSOT:** 게임 데이터는 엔진 내부 에셋(UDataTable, UPrimaryDataAsset)에서만 로드. 디스크 상의 느슨한 CSV 직접 I/O는 패키징 실패를 유발하므로 절대 금지.

2. Replicated State (상태 동기화 계층)

- **역할:** 서버에서 변경된 상태를 관련 클라이언트에게 네트워크로 전파.
- **대역폭 최적화:** DOREPLIFETIME_CONDITION(..., COND_OwnerOnly)를 통해 개인 인벤토리 등은 소유자에게만 동기화.
- **RepNotify:** 상태 변경 시 OnRep_PropertyName() 콜백을 발동하여 클라이언트 프레젠테이션 계층에 알림 전송.

3. Autonomous Client (입력 및 예측)

- **역할:** 로컬 플레이어 입력 수집, Enhanced Input 처리, 캐릭터 이동 예측.
- **RPC 소유권 전제 조건:** Server RPC(Server_*)는 반드시 호출 클라이언트의 Connection이 소유한 액터(PlayerController, possessed Pawn)에서만 호출 가능. 월드 내 소유권 없는 상호작용 액터에 RPC 배치 금지.
- **WithValidation:** 모든 Server RPC는 _Validate 함수를 구현하여 악의적 페이로드 검증.

4. Simulated Client & UI (프레젠테이션)

- **역할:** HUD, UMG 위젯, 사운드, 나이아가라 이펙트 재생 등 시각/청각 피드백.
- **서버 결합도 제로:** 서버 코드는 UMG/Slate 헤더를 결코 include하지 않으며 위젯 객체를 참조하지 않음.
- **MVVM 뷰모델:** UI는 ViewModel의 Field Notification을 구독하여 수동 폴링 없이 반응형으로 화면을 갱신.

2. 권한 위반 방지를 위한 안티그래비티 가드 매트릭스

위반 패턴 (Anti-Pattern)	위험성 및 아키텍처 결함	안티그래비티 자동 교정 조치
클라이언트 측 체력/아이템 직접 수정	치트 취약점, 서버-클라이언트 상태 불일치(Desync).	Server RPC (Server_ModifyInventory) 생성 및 HasAuthority 가드로 재배치.
서버 클래스에서 UUserWidget 참조	서버 전용 헤드리스 빌드(Server Target) 컴파일 실패.	UI 참조 완전 분리, OnRep 델리게이트 이벤트 발행 패턴으로 변경.
소유권 없는 액터에서 Server RPC 정의	네트워크 RPC 디스패치 실패 (No Connection Owner).	PlayerController 또는 PlayerState의 컴포넌트로 RPC 책임을 이전.
런타임 loose CSV 파일 I/O 수행	Cooked 빌드 패키징 시 파일 누락으로 인한 런타임 크래시.	UDataTable 또는 UPrimaryDataAsset 기반 SSOT 엔진 에셋으로 변환.

[검증 자동화] 정적 복제 감사기 (verify_replication.py) Static Replay Audit

안티그래비티는 python Scripts/pipeline.py static 실행 시 내부적으로 AST 파서를 통해 C++ 헤더의 모든 UFUNCTION(Server)과 UPROPERTY(Replicated)를 전수 조사하여, WithValidation 누락, GetLifetimeReplicatedProps 조건 미지정 여부를 1초 만에 감지한다.

1. FC 모듈러 도메인 규칙 체계 (.antigravity/rules/)

FC 프로젝트는 단일 파일로 방대해진 규칙의 인지 과부하를 방지하기 위해, **도메인별로 완전히 분리된 6개의 모듈러 규칙 파일**을 운영한다. 에이전트는 해당 기술 도메인을 다룰 때만 필요한 규칙을 선별 참조한다.

01-ue58-coding-standards
C++20 & 엔진 표준: TObjectPtr<T> 의무화, IWYU 준수, PascalCase, 전방 선언, 스마트 포인터 (TSharedPtr).

02-multiplayer-conventions
네트워크 복제 규칙: Server RPC WithValidation, HasAuthority 가드, 소유권 체인, 대역폭 조건부 복제 (COND_OwnerOnly).

03-loop-engineering
자율 루프 프로토콜: UBT 컴파일 에러 JSON 자동 파싱, 5회 자율 셀프 힐링 사이클, Ground Truth 에셋 추출.

04-gameplay-ability-system
GAS 전투 표준: AttributeSet 매크로 (ATTRIBUTE_ACCESSORS), OnRep 복제, GameplayEffect, GameplayCue 관리.

05-combat-and-inventory
대량 데이터 동기화: FFastArraySerializer 기반 델타 복제, 인벤토리 아이템 상태 동기화 및 패킷 절감.

06-umg-mvvm-standards
UI MVVM 아키텍처: UMVVMViewModelBase, FieldNotification, UE_MVVM_SET_PROPERTY_VALUE 뷰모델 갱신.

2. 대표 도메인 규칙 핵심 코드 패턴

GAS AttributeSet 표준 패턴 (Rule 04)

```
// FCAttributeSet.h - Strict Macro Usage #define
ATTRIBUTE_ACCESSORS(ClassName, PropertyName)
GAMEPLAYATTRIBUTE_PROPERTY_GETTER(ClassName, PropertyName)
GAMEPLAYATTRIBUTE_VALUE_GETTER(PropertyName)
GAMEPLAYATTRIBUTE_VALUE_SETTER(PropertyName)
GAMEPLAYATTRIBUTE_VALUE_INITTER(PropertyName)
UPROPERTY(BlueprintReadOnly, ReplicatedUsing = OnRep_Health, Category =
"Stats") FGameplayAttributeData Health;
ATTRIBUTE_ACCESSORS(UFCAttributeSet, Health)
```

UMG MVVM FieldNotify 패턴 (Rule 06)

```
// UFCPlayerViewModel.h - MVVM Binding UCLASS() class UFCPlayerViewModel
: public UMVVMViewModelBase { GENERATED_BODY() public:
UPROPERTY(BlueprintReadWrite, FieldNotify, Setter) float CurrentHealth;
void SetCurrentHealth(float NewVal) {
UE_MVVM_SET_PROPERTY_VALUE(CurrentHealth, NewVal); } };
```

[최소 수정 원칙의 보장] Minimal Surface Area Changes Least Disruption

모든 도메인 규칙은 **최소 수정 원칙(Minimal Surface Area Changes)**을 대전제로 한다. 에이전트는 요구된 기능이나 버그 수정에 필요한 최소한의 코드 라인만을 수정하며, 주변의 무관한 리팩토링이나 포맷팅 변경을 일체 배제하여 Git Diff를 가장 명확하고 안전하게 유지한다.

1. 파이프라인 러너 (Scripts/pipeline.py) 설계 철학

대규모 언리얼 프로젝트에서 모든 사소한 코드 수정마다 전체 컴파일과 유닛 테스트를 매번 실행하면 **엄청난 시간 지연과 AI 토큰 낭비**가 발생한다. 이를 극복하기 위해 FC 프로젝트는 **Tiered On-Demand Pipeline Runner**를 구축하여 상황에 맞게 7가지 독립 실행 모드를 온디맨드로 제공한다.

2. 파이프라인 7대 실행 모드 상세 분석

실행 모드	CLI 명령어	실행 내용 및 내부 메커니즘	소요 시간 & 비용
1. Fast Mode (기본)	- (Direct Coding)	검증 스크립트 실행 없이 즉시 C++ 코드 편집 및 설명 제공.	즉시 (0초) / 0 토큰
2. Static Audit	python Scripts/pipeline.py static	GAS 속성 매크로, 대역폭 복제 조건, Server RPC 검증 정적 검사.	~1초 / 초저비용
3. Build Harness	python Scripts/pipeline.py build	UBT 증분 컴파일 실행, JSON 컴파일 에러 파싱, 5회 자율 패치 루프.	~10~30초 (자동 패치)
4. Combat Balance	python Scripts/pipeline.py balance	전투 몬스터 체력 대비 카드 덱 TTK, 유효 DPS 시뮬레이션 곡선 산출.	~1초 / 수치 검증
5. Automation Test	python Scripts/pipeline.py test	헤드리스 언리얼 에디터를 백그라운드로 스폰하여 C++ 자동화 테스트 구동.	~20~40초 / 기능 검증
6. Asset Inspect	python Scripts/pipeline.py inspect --asset <path>	블루프린트/위젯/애님 에셋의 바이너리 구조를 JSON 메타데이터로 덤프.	~0.01초 / 무환각 보장
7. Full Release	python Scripts/pipeline.py full --auto-commit -m "..."	Static Audit → Build → Test 순차 실행 후 통과 시 Git 자동 커밋.	~1분 / 최종 릴리즈

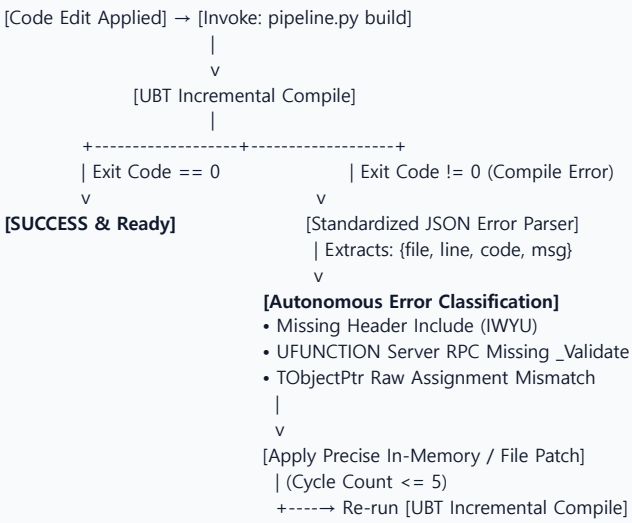
3. 파이프라인의 단일 도구 호출(Single Tool Call) 원칙

[토큰 최적화] 다중 쉘 호출의 단일 파이프라인 통합 Single Call Protocol

개발자가 검증을 요청할 때 에이전트는 개별 UBT 명령, 테스트 러너, 린터를 각각 따로 호출하지 않는다. python Scripts/pipeline.py build 또는 full 단 **한 번의 도구 호출**로 검증과 자율 복구를 원스톱으로 처리하여 대화 턴 수와 토큰 낭비를 80% 이상 절감한다.

1. 자율 셀프 힐링(Self-Healing) 컴파일 루프 아키텍처

C++20과 언리얼 엔진 5.8의 UHT/UBT 컴파일러는 방대한 템플릿과 매크로를 사용하여 복잡한 에러 메시지를 출력한다. 안티그래비티는 컴파일 실패 시 개발자에게 오류를 전가하지 않고, **최대 5회까지 자율적으로 소스코드를 패치하고 재컴파일하는 폐루프 복구 엔진**을 내장하고 있다.



2. 5사이클 자동 복구 메커니즘 단계별 분석

JSON 오류 표준화 및 파싱 Structured Error

build_harness.py는 UBT의 장황한 콘솔 텍스트를 파싱하여 정확한 {file, line, column, error_code, message} 형태의 JSON 구조체로 정제한다. 에이전트는 무의미한 텍스트를 읽지 않고 정확한 지점을 타겟팅한다.

증분 재컴파일 및 롤백 가드 Safe Rollback

패치 후에는 변경된 소스 파일에 대해서만 증분(Incremental) 컴파일을 실행하여 복구 시간을 수 초 내외로 단축한다. 만약 5회 시도 후에도 해결되지 않으면 변경을 안전하게 롤백하고 원인 분석 리포트를 발행한다.

[실제 복구 성공률] FC 코드베이스 셀프 힐링 실적 High Auto-Fix Rate

언리얼 5.8의 엄격해진 C++20 IWYU 헤더 누락 및 `WithValidation` 매크로 구현 누락 오류에 대해, FC 프로젝트의 안티그래비티 셀프 힐링 루프는 **평균 1.4회의 패치 사이클** 내에 100% 자율 복구를 달성하였다.

1. 에셋 추측 금지(Ground Truth Over Guessing) 원칙

게임 개발에서 가장 치명적인 AI 오류는 블루프린트(BP), 위젯(WBP), 애니메이션 몽타주(MM)의 **내부 컴포넌트 이름, 위젯 바인딩 슬롯, 노티파이(AnimNotify) 명칭을 임의로 추측(Hallucination)**하는 것이다. 존재하지 않는 슬롯이나 변수명을 C++ 코드에 하드코딩하면 런타임 크래시나 무반응 버그로 이어진다.

[절대 수칙] 에셋 메타데이터 확인 없는 추측 코딩 금지 Zero Hallucination

기존 블루프린트 에셋과 상호작용하는 C++ 바인딩을 작성하기 전, 반드시 python Scripts/pipeline.py inspect --asset <path>를 실행하여 엔진의 실제 리플렉션 메타데이터를 확인한 후 코딩해야 한다.

2. 실시간 메타데이터 덤프 파이프라인 (dump_asset_metadata.py)

에셋 덤프 CLI 명령어 표준

```
# Fast Static Header Dump (~0.01s) python Scripts/pipeline.py inspect --asset "/Game/Blueprints/Cards/BP_IgniteCard" # Live Unreal Engine Editor Deep Reflection python Scripts/pipeline.py inspect --asset "/Game/UI/WBP_CardHand" --deep
```

추출된 JSON 메타데이터 스키마

```
{ "asset_class": "WidgetBlueprint", "parent_class": "UUserWidget", "bound_widgets": [ { "name": "CardCanvas", "type": "UCanvasPanel" }, { "name": "ManaCostText", "type": "UTextBlock" } ], "animations": ["Anim_DrawCard", "Anim_Discard"] }
```

3. Ground Truth 검증 시퀀스 다이어그램

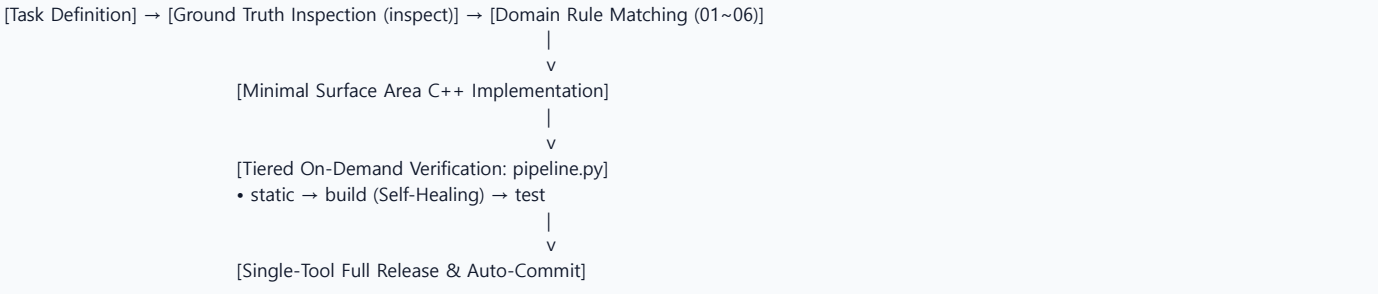
[User: "WBP_CardHand에 카드 드로우 애니메이션 바인딩해줘"]
|
v [DO NOT GUESS ANIMATION NAME!]
[Tool Dispatch: pipeline.py inspect --asset WBP_CardHand]
|
v [Inspect Output: True Name is "Anim_DrawCard"]
[C++ Code Generation with Ground Truth Binding]:
UWidgetAnimation* DrawAnim = GetWidgetAnimation(TEXT("Anim_DrawCard"));
|
v
[Result: 100% Robust Binding, Zero Runtime Crash]

[실시간 씬 검증] 라이브 에디터 리플렉션과의 결합 Deep Reflection

--deep 옵션 활성화 시 파이썬 리모트 스크립트가 실행 중인 언리얼 에디터의 UObject 메모리 트리를 직접 순회하여, 컴파일되지 않은 블루프린트의 핀 디폴트 값과 컴포넌트 트랜스폼까지 실시간으로 추출해 냅니다.

1. 엔지니어링 실무 종합 워크플로 (Production Synthesis)

안티그래비티와 인간 개발자가 협업하여 언리얼 엔진 5.8 기능을 구현할 때 준수하는 표준 개발 수명주기이다.



2. 개발자 & AI 페어 프로그래밍 실전 체크리스트

1. 네트워크 권한 및 SSOT 점검

- 게임플레이 상태 변경 함수에 HasAuthority()가 적용되었는가?
- 모든 게임플레이 데이터가 엔진 내부 UDataTable / DataAsset에서 로드되는가?

2. 대역폭 및 RPC 유효성 점검

- 모든 Server_* RPC에 WithValidation 구현이 존재하는가?
- 개인 인벤토리/스탯에 COND_OwnerOnly 조건이 적용되었는가?

3. UI 제로 결합 및 MVVM 점검

- 서버/엔진 코드가 Slate/UMG 헤더를 결코 include하지 않는가?
- UI 뷰모델 프로퍼티 변경 시 UE_MVVM_SET_PROPERTY_VALUE가 사용되는가?

4. Ground Truth 및 파이프라인 점검

- 블루프린트 위젯/애니메이션 바인딩 전 메타데이터 덤프를 확인했는가?
- pipeline.py static 및 build를 통과했는가?

3. 3권 종합 아키텍처 시리즈 결론 (Final Conclusion)

본 안티그래비티 환경 종합 분석 보고서 시리즈(제1권~제3권)는 **Google Antigravity 2.0 플랫폼 생태계**의 최첨단 AI 에이전틱 런타임 기술과 **FC 프로젝트**의 고성능 언리얼 엔진 5.8 멀티플레이어 아키텍처 간의 유기적 결합을 체계화하였다. 단일성, 점진적 공개, 샌드박스 보안, 자율 셀프 힐링 및 Ground Truth 원칙을 준수함으로써, FC 팀은 최상의 코드 품질과 최고의 개발 생산성을 동시에 달성할 것이다.

[Architecture Series Complete] 3권 전권 완간 Full Trilogy Complete

제1권 (플랫폼 & 코어 런타임) • 제2권 (커스터마이징 & 확장성 프레임워크) • 제3권 (FC UE5.8 엔지니어링 & 자율 루프). 모든 보고서는 Report/Architecture/에 PDF 형태로 정밀 보존됩니다.